Space Courier: схема данных

Oct 24, 2026 · @Андрей

Вся игра описывается данными в JSON: движок на Godot один раз читает файлы, а новые заказы, сцены и героинь добавляешь записями без правки кода.

Структура проекта

Сцены отвечают за то, как показывать, а данные за то, что показывать. Сцены пишутся один раз, данные растут записями.

```
res://
  data/
    heroines/    pilot.json, medic.json, security.json, mechanic.json,
boss.json
    scenes/      одна запись = одна сцена (текст, условия, последствия)
    locations/   hub.json, ship.json, schedules.json
    orders/      private.json, state.json, crime.json
    galaxy/      systems.json, routes.json, events.json
    ship/        ship.json, modules.json
    combat/      enemies.json, abilities.json
  scenes/        hub, ship, galaxy_map, system_flight, combat, dialogue
  scripts/       GameState, DataLoader, Clock, ConditionEvaluator,
EffectApplier
  art/           characters/<id>/ (base, face, blush), backgrounds/
```

Модуль	Что делает
GameState (autoload)	хранит всё состояние: шкалы, флаги, деньги, топливо, известные маршруты, день и время; сохранение игры = сериализация этого объекта
DataLoader	читает JSON из data/ при старте и проверяет ссылки между файлами (id сцены, героини, локации)
Clock	день и слот времени (утро, день, вечер, ночь), сигналы при смене
ConditionEvaluator	проверяет список условий из JSON
EffectApplier	применяет список последствий из JSON

Все системы (сцены, заказы, события, бой) говорят на одном языке условий и последствий (см. раздел «Сцены»), поэтому новый контент не требует нового кода. Id пишутся латиницей в snake_case, ссылки между файлами идут только по id.

Героиня

У каждой героини три шкалы от 0 до 100. Отношение решает, что вообще возможно; настроение и возбуждение меняют окраску сцен, внешний вид и бонусы.

Шкала	Скорость	Роль
attitude (отношение)	медленная	ворота: пороги открывают уровни взаимодействия
mood (настроение)	средняя	множитель изменений, лицо спрайта, бонусы в бою
arousal (возбуждение)	быстрая, спадает	вариации реплик и сцен, румянец; работает только выше порога touch

Стартовые значения для баланса: arousal падает на 25 при каждом сдвиге времени суток и обнуляется после сцены; при mood от 70 рост attitude умножается на 1,25, при mood до 30 на 0,75, а падение работает наоборот.

```
{
  "id": "pilot",
  "name": "Имя",
  "role": "pilot",
  "stats": { "attitude": 20, "mood": 60, "arousal": 0 },
  "thresholds": { "talk": 0, "touch": 30, "kiss": 55, "intimate": 75 },
  "tag_weights": { "smuggling": 3, "risk": 2, "law_abiding": -2, "betrayal":
-5 },
  "polygamy": "tolerates",
  "suspicion": 0,
  "contact_block": false,
  "aboard": true,
  "schedule": "pilot",
  "sprite": {
    "bases": ["uniform", "casual"],
    "faces": ["neutral", "happy", "sad", "annoyed", "shy"],
    "blush_levels": 3
  },
  "abilities": ["evasive_maneuver", "emergency_burn"]
}
```

Поле	Смысл
thresholds	значения attitude, с которых открываются уровни: разговор, прикосновения, поцелуи, близость. У каждой героини свои
tag_weights	что она думает о поступках: плюс нравится, минус нет. Когда она на борту или рядом, действие игрока с тегом меняет attitude на вес тега, умноженный на силу действия
polygamy	accepts, tolerates или forbids
suspicion	шкала подозрений 0-100, растёт от свидетелей, следов и болтливой третьей героини
contact_block	блок связи после раскрытия

Секретность (гарем)

- forbids: при suspicion от 100 запускается сцена раскрытия, attitude падает на 40 и ставится contact_block. Снимается квестом примирения или разрывом с остальными.
- tolerates: ревность снижает mood, лечится подарком или отдельной сценой.

- accepts: может прикрывать, снижая suspicion других на 10 за сутки.

Спрайт слоями. Движок собирает картинку по шкалам: лицо берётся по настроению (от 70 happy, 40-69 neutral, 20-39 annoyed, ниже 20 sad), румянец по возбуждению (до 24 нет, 25-59 уровень 1, от 60 уровень 2), и румянец показывается только если attitude не ниже порога touch. Одна база позы плюс набор лиц и оверлеев вместо сотен отдельных картинок.

Сцены

Сцена это запись с условиями, репликами, выбором и последствиями. Интимные сцены ничем не отличаются по формату: у них тег intimacy и пороги отношения в условиях.

```
{
  "id": "pilot_hug_01",
  "type": "ship",
  "tags": ["romance", "touch"],
  "location": "ship_bridge",
  "conditions": [
    { "stat": "pilot.attitude", "op": ">=", "value": 30 },
    { "stat": "pilot.mood", "op": ">=", "value": 50 },
    { "time": ["evening", "night"] },
    { "others_absent": ["medic", "security"] },
    { "flag": "pilot_hug_01_seen", "is": false }
  ],
  "lines": [
    { "who": "pilot", "face": "shy", "text": "Реплика героини" },
    { "who": "hero", "text": "Реплика героя" }
  ],
  "choices": [
    { "id": "stay", "text": "Остаться", "effects": [
      { "stat": "pilot.attitude", "add": 3 },
      { "stat": "pilot.arousal", "add": 20 }
    ] },
    { "id": "leave", "text": "Уйти", "effects": [
      { "stat": "pilot.attitude", "add": -1 }
    ] }
  ],
  "on_end": [
    { "flag": "pilot_hug_01_seen", "set": true },
    { "advance_time": 1 }
  ],
  "next": ["pilot_kiss_01"]
}
```

Условия (все должны выполняться):

Тип	Пример	Смысл
stat	`{"stat":"pilot.attitude","op":">=","value":30}`	сравнение шкалы
flag	`{"flag":"x","is":false}`	флаг сюжета
time	`{"time":["evening"]}`	слот суток
location	`{"at":"ship_bridge"}`	где игрок
others_absent	`{"others_absent":["medic"]}`	этих героинь нет в локации
aboard	`{"aboard":"mechanic","is":true}`	кто на борту
rep	`{"rep":"mafia","op":"<","value":-20}`	репутация

Последствия (применяются по порядку):

Тип	Пример
stat	`{"stat":"pilot.mood","add":5}`
flag	`{"flag":"x","set":true}`
rep, money, fuel	`{"rep":"state","add":-3}`, `{"money":500}`, `{"fuel":-2}`
item	`{"item":"gift_flower","add":1}`
suspicion	`{"suspicion":{"target":"others","add":10}}`
time	`{"advance_time":1}`
unlock	`{"unlock_route":"r_alpha_beta"}`, `{"unlock_scene":"x"}`, `{"start_order":"ord_001"}`

Этот набор расширяется новыми типами в ConditionEvaluator и EffectApplier, а сами сцены остаются данными.

Локации и расписание

В сутках четыре слота: утро, день, вечер, ночь. Каждое действие тратит слоты, а по слоту расписание решает, кто где находится.

Зона	Локации
Хаб	hub_kitchen, hub_room_<героиня>, hub_showers, hub_tech, hub_office, hub_shop
Корабль	ship_bridge, ship_medbay, ship_cabin_<героиня>, ship_cargo

```
{
  "id": "hub_kitchen",
  "area": "hub",
  "name": "Гостиная и кухня",
  "backgrounds": { "day": "bg/kitchen_day.png", "night":
"bg/kitchen_night.png" },
  "connects": ["hub_corridor"],
  "access": [],
  "actions": ["talk", "gift", "wait"]
}
```

```
{
  "heroine": "mechanic",
  "slots": {
    "morning": [{ "loc": "hub_tech", "weight": 1 }],
    "day": [{ "loc": "hub_tech", "weight": 1 }],
    "evening": [{ "loc": "hub_kitchen", "weight": 2 }, { "loc": "hub_tech",
"weight": 1 }],
    "night": [{ "loc": "hub_room_mechanic", "weight": 1 }]
  },
  "overrides": [
    { "if": [{ "aboard": "mechanic", "is": true }], "slots": { "day": [{
"loc": "ship_cargo", "weight": 1 }] } }
  ]
}
```

Место выбирается по весам при смене слота и фиксируется до следующей смены, чтобы игрок мог ей пользоваться. Если weight больше одной записи, героиня может оказаться в любой из них.

Рискованные действия (заглянуть, взять чужое) описываются так же данными:

```
{
  "id": "take_item_crew_room",
  "location_tag": "crew_room",
  "stealth": { "difficulty": 2, "rounds": 1 },
  "on_success": [{ "item": "personal_item", "add": 1 }],
  "on_caught": { "scene": "caught_in_room", "effects": [{ "stat":
"owner.attitude", "add": -5 }] }
}
```

Реакция при поимке выбирается по характеру героини: у каждой своя сцена поимки, поэтому в данных стоит заглушка owner.

Стелс-мини-игра

Подглядывание, кража вещей и другие скрытые действия проходят через мини-игру на тайминг. Значок едет по линии туда и обратно, игрок нажимает ЛКМ или пробел, и если в этот момент значок на зелёной зоне, действие удалось. Мимо зоны запускает on_caught: сцену поимки с реакцией героини.

Зелёная зона растёт с количеством пройденных стелс-действий и сужается от сложности. Стартовые значения для баланса:

```
zone  = clamp(0.12 + min(0.25, stealth_passed × 0.01) − difficulty × 0.03,
0.04, 0.45)
speed = 0.8 + difficulty × 0.15   (проходов линии в секунду)
```

Параметр	Откуда	Эффект
stealth_passed	GameState, +1 за каждое успешное стелс-действие	расширяет зону, максимум +0,25
difficulty	поле stealth.difficulty у действия, от 1 до 5	сужает зону на 0,03 за единицу и ускоряет значок
rounds	поле stealth.rounds	сколько попаданий подряд нужно; для сложных действий 2-3
модификаторы	расписание и состояние героини	другая героиня в локации +1 к difficulty, ночь −1, suspicion владелицы выше 50 даёт +1

В сценах мини-игра запускается обычным последствием, поэтому подходит любому выбору игрока:

```
{ "text": "Заглянуть в щель", "effects": [
  { "start_stealth": { "difficulty": 3, "rounds": 2, "on_success":
"shower_peek_ok", "on_fail": "shower_peek_caught" } }
] }
```

- Мини-игра это отдельная Godot-сцена scenes/stealth_minigame. Она получает difficulty и rounds, возвращает успех или провал и ничего не знает о контенте.
- Успех добавляет 1 к stealth_passed. Провал ничего не отнимает, а ведёт на сцену on_fail, которая у каждой героини своя.
- Провал не заканчивает игру: поимка даёт изменение отношения, подозрения или сцену.
- В настройки полезно добавить режим «замедлить значок» для удобства.

Магазин, готовка и предметы

Лёгкие механики хаба, которые связывают полёты с отношениями: то, что игрок находит в космосе, он тратит на героинь. Сюжеты героинь пока не написаны, поэтому все эти системы работают на тестовых данных-заглушках. Новые файлы: data/items/items.json, data/shops/shops.json, data/recipes/recipes.json.

Предметы. Все предметы лежат в одном файле, на них ссылаются заказы, рецепты и магазины:

```
{
  "id": "xeno_crystal",
  "name": "Кристалл с окраины",
  "type": "quest",
  "tags": [],
  "mass": 1,
  "sell_price": 0
}
```

Типы: quest (для поручений), ingredient (для готовки), gift (подарок), loot (на продажу), dish (готовое блюдо). В условия добавляется проверка `{"item": "xeno_crystal", "count": 1}`, последствия уже умеют `{"item": "xeno_crystal", "add": -1}`. Масса предмета занимает место в трюме.

Находки в полётах. В objects системы добавляется тип salvage. Подлёт запускает сбор, предметы выпадают по шансам и попадают в трюм:

```
{ "type": "salvage", "pos": [640, 280], "once": true,
  "loot": [ { "item": "xeno_crystal", "chance": 0.3 }, { "item":
"space_spice", "chance": 0.6 } ] }
```

Магазин (hub_shop). Новая локация хаба по соседству. Продавщица это ксено-героиня shopkeeper вне команды: она не живёт в хабе и не бывает на борту, игрок видит её только в магазине. Расписание ей не нужно: она на месте все слоты, когда магазин открыт (open_slots).

```
{
  "id": "hub_shop_stock",
  "owner": "shopkeeper",
  "open_slots": ["day", "evening"],
  "items": [
    { "item": "noodles", "price": 12 },
    { "item": "hot_sauce", "price": 18 },
    { "item": "cookbook_xeno_spices", "price": 250, "min_attitude": 20 },
    { "item": "gift_flower", "price": 60, "min_attitude": 10 }
  ],
  "buys": ["loot", "ingredient"],
  "sell_ratio": 0.5
}
```

Поле min_attitude открывает товар по отношению продавщицы. Игрок покупает еду, ингредиенты и подарки и продаёт находки по sell_ratio от цены.

Короткая линия продавщицы состоит из 3-4 поручений. Это обычные заказы из раздела «Заказы и репутация» с источником shop, но без дедлайна и штрафа за провал:

```
{
  "id": "shop_quest_02",
  "source": "shop",
  "giver": "shopkeeper",
  "title": "Принести кристалл из дальних систем",
  "requires": [
    { "stat": "shopkeeper.attitude", "op": ">=", "value": 10 },
    { "flag": "shop_quest_01_done", "is": true }
  ],
  "need_items": [{ "item": "xeno_crystal", "count": 1 }],
  "reward": [
    { "stat": "shopkeeper.attitude", "add": 15 },
    { "flag": "shop_quest_02_done", "set": true },
    { "unlock_scene": "shopkeeper_story_02" }
  ]
}
```

- Два типа поручений: fetch (принести предмет из need_items) и labor (помочь в магазине, поле labor_slots забирает слоты времени и предметов не требует).

- Награда это рост attitude и открытие следующей сцены. Финал цепочки открывает близость по обычным порогам thresholds героини.
- Сами сцены остаются заглушками, пока не будет сюжета.

Готовка (hub_kitchen). Рецепт превращает ингредиенты в блюдо. Блюдо можно съесть самому или подать героине как ухаживание.

```
{
  "id": "recipe_spicy_noodles",
  "name": "Острая лапша",
  "ingredients": [ { "item": "noodles", "count": 1 }, { "item": "hot_sauce",
"count": 1 } ],
  "slots": 1,
  "tags": ["spicy", "hot"],
  "self_effects": [{ "buff": "steady_hands", "days": 1 }],
  "serve_effects": [
    { "stat": "{target}.mood", "add": 6 },
    { "stat": "{target}.attitude", "add": 2 }
  ]
}
```

У героини в её JSON появляется поле `"food": { "likes": ["spicy"], "dislikes": ["sweet"] }`. Новый тип последствия buff даёт временный бонус на дни, например steady_hands добавляет +0,05 к зелёной зоне стелс-мини-игры. Стартовые значения для баланса:

```
quality = min(3, 1 + floor(cooking_skill ÷ 5))     (cooking_skill +1 за
каждое приготовленное блюдо)
gain    = base × (1 + 0.25 × (quality − 1)) × taste
taste   = 1.5 если тег блюда в likes, 1.0 нейтрально, −1.0 если в dislikes
```

- Готовка стоит слот времени и ингредиенты, поэтому вечер на ужин это выбор против других героинь и вылазок.
- Подать блюдо можно раз в сутки одной героине и только если её mood не ниже 20, иначе она отказывается.
- Подача на кухне публична: если рядом героини с polygamy forbids, их suspicion растёт на 5, потому что ухаживание заметно.
- Интерфейс кухни: список рецептов, проверка ингредиентов в инвентаре, результат кладётся в инвентарь как блюдо с полем quality, дальше действие serve у героини в локации.

Кулинарные книги. Начальный список рецептов маленький, а остальные рецепты открываются книгами, которые можно найти или купить.

```
{
  "id": "cookbook_xeno_spices",
  "name": "Кулинария дальних миров",
  "type": "cookbook",
  "mass": 1,
  "sell_price": 0,
  "teaches": ["recipe_crystal_tea", "recipe_xeno_stew"]
}
```

- В recipes.json у базовых рецептов стоит `"starter": true`, они известны с начала игры. Остальные нужно открыть книгой.
- Книгу можно прочитать из инвентаря. Новое последствие `{"learn_recipes": [...]}` добавляет рецепты в known_recipes в GameState, и рецепты остаются навсегда. Сама книга убирается из трюма и освобождает место.
- Источники книг: магазин (часть открывается по min_attitude продавщицы), обломки в системах (редкие книги в дальних системах), награды за поручения и подарки героинь.
- Рецепты из дальних книг могут требовать редкие ингредиенты и давать редкие теги вкуса, поэтому исследование карты и ухаживание связаны между собой.
- DataLoader проверяет при старте, что у каждого нестартового рецепта есть хотя бы одна книга, которая его учит, и что у каждой книги есть источник (магазин, обломки или награда).

Заказы и репутация

Заказ берётся с ПК в кабинете начальницы. Источник определяет награду, риск и последствия провала.

Источник	Награда	Риск	При провале
private	средняя	низкий	денежный штраф
state	средняя + репутация state	бывает опасный район	падает репутация state (социальный кредит)
crime	самая высокая	нелегальный груз, проверки патруля, опасные районы	флаг mafia_hunt: мафия охотится за героем

```
{
  "id": "ord_state_food_017",
  "source": "state",
  "title": "Продовольствие в дальнюю колонию",
  "cargo": { "type": "food", "mass": 4, "tags": ["legal"] },
  "from": "sys_earth",
  "to": "sys_colony_3",
  "deadline_days": 9,
  "requires": [{ "rep": "state", "op": ">=", "value": 0 }],
  "reward": [{ "money": 1200 }, { "rep": "state", "add": 2 }],
  "failure": [{ "rep": "state", "add": -4 }],
  "action_tags": ["law_abiding", "help_civilians"],
  "risk_tags": ["dangerous_district"]
}
```

- Репутаций три: state, mafia, private, диапазон от -100 до 100. Они открывают и закрывают заказы через условие rep.
- action_tags при выполнении заказа попадают в tag_weights героинь, которые на борту. Контрабанда обрадует пилота и расстроит безопасницу.
- Провал никогда не заканчивает игру: он запускает последствия в виде штрафа, события погони или сцены с командой. Например, флаг mafia_hunt включает в пул событий систем сцену с преследователями.
- Дедлайн считается в днях, а не в реальном времени.

Карта галактики

Галактика это граф: системы-узлы и маршруты-рёбра. После Варп-шторма часть маршрутов неизвестна, и игрок открывает их сам.

Туман войны и связи между системами

Большая часть галактики скрыта туманом. Для каждой системы в GameState хранится состояние известности:

Состояние	Что видит игрок	Как получить
hidden	ничего, туман	по умолчанию
seen	значок и название системы, пути из неё неизвестны	цель заказа, купленная карта, сканер
visited	система и все пройденные из неё пути	прилёт

Связи между системами расставлены вручную в routes.json и не следуют географии. Путь из соседней со стартом системы может вести на противоположный край галактики, а оттуда нельзя прыгнуть к соседям старта. Поэтому рёбра направленные: обратный путь это отдельная запись, и его может не быть.

```
{
  "id": "r_near2_far9",
  "from": "sys_near_2",
  "to": "sys_far_9",
  "directed": true,
  "fuel": { "min": 5, "max": 14, "true": 9 },
  "days": 3,
  "hazard": 0.5,
  "known": false,
  "public": false,
  "map_value": 120
}
```

Карта это предмет, который открывает системы и маршруты:

```
{ "id": "map_far_east", "price": 600, "reveals_systems": ["sys_far_9",
"sys_far_10"], "reveals_routes": ["r_far9_far10"] }
```

- Поиск пути идёт по известным направленным рёбрам. Если у системы нет известного выхода, UI пишет «обратный путь неизвестен».
- DataLoader при старте проверяет, что из каждой системы хаб достижим хотя бы одним путём, пусть очень дорогим. Иначе игрок застрянет без выхода. Дальний край с единственным долгим возвращением делает исследование рискованным, но не безвыходным.
- Туман рисуется слоем поверх систем со статусом hidden. Для seen показывается только сама система, для visited ещё и известные из неё рёбра.

```
{
  "id": "sys_alpha",
  "name": "Альфа",
  "map_pos": [120, 340],
  "danger": 2,
  "refuel": { "available": true, "price_per_unit": 40, "crime_discount_rep":
"mafia" },
  "event_pool": ["pirates_small", "patrol_check", "distress_signal"],
  "objects": [
    { "type": "planet", "id": "alpha_1", "pos": [300, 220] },
    { "type": "gate", "to_route": "r_alpha_beta", "pos": [820, 540] },
    { "type": "event_marker", "event": "distress_signal", "pos": [500, 700] }
  ]
}
```

```
{
  "id": "r_alpha_beta",
  "from": "sys_alpha",
  "to": "sys_beta",
  "fuel": { "min": 3, "max": 6, "true": 4 },
  "days": 1,
  "hazard": 0.2,
  "known": false,
  "public": false,
  "map_value": 50
}
```

- **Неизвестные пути.** Игроку показывается только диапазон min-max, а true задаётся в данных вручную и открывается после первого прохода.
- **Планирование.** Маршрут по известным рёбрам ищется алгоритмом Дейкстры. Ожидаемый расход считается по середине диапазона, и игра предупреждает, если топлива может не хватить на возвращение.
- **Карты как товар.** Пройденный путь попадает в known и в нарисованную игроком карту. Продажа карты даёт money по map_value, но ставит public: true, и мафия с патрулями начинают использовать этот путь в событиях.
- **Полёт в системе.** Сцена system_flight показывает объекты из objects, работает в реальном времени, как в «Космических рейнджерах»: корабль игрока и другие корабли системы летят одновременно, пауза по пробелу остаётся как удобство. Подлёт к event_marker запускает событие из events.json, подлёт к gate переход по маршруту.

```
{
  "id": "patrol_check",
  "trigger": "enter_system",
  "chance": 0.35,
  "conditions": [{ "cargo_has_tag": "contraband" }],
  "options": [
    { "text": "Откупиться", "effects": [{ "money": -400 }], "tags": ["bribe"]
},
    { "text": "Бежать", "effects": [{ "fuel": -3 }, { "start_combat":
"patrol_light" }], "tags": ["risk"] },
    { "text": "Сдаться", "effects": [{ "rep": "state", "add": -2 }, {
"money": -800 }], "tags": ["law_abiding"] }
  ]
}
```

Каждый вариант события стоит топлива, денег, репутации или чьей-то симпатии, и поэтому выбор что-то значит.

Корабль

```
{
  "id": "ship_main",
  "hull": { "max": 100, "current": 100 },
  "fuel": { "max": 30, "current": 30 },
  "cargo": { "max_mass": 10, "items": [] },
  "slots": { "engine": 1, "weapon": 2, "utility": 2 },
  "modules": ["engine_mk1", "gun_light", "shield_basic"],
  "crew_aboard": ["pilot", "medic", "security"]
}
```

```
{
  "id": "tank_ext_1",
  "slot": "utility",
  "cost": 800,
  "effects": [{ "ship": "fuel.max", "add": 10 }],
  "requires": [{ "stat": "mechanic.attitude", "op": ">=", "value": 20 }]
}
```

Расход топлива (стартовые значения для баланса): стоимость прыжка равна true маршрута, умноженному на (1 + 0,05 × масса груза), с округлением вверх. Форсаж добавляет 50 процентов. Когда топливо на нуле, это не конец игры: срабатывает событие «сигнал бедствия, буксир», игрок получает долг начальнице или мафии.

Ремонт и механик

- Пока флаг mechanic_aboard не установлен, полный ремонт и установка модулей возможны только в хабе (hub_tech).
- В полёте доступен аварийный ремонт: восстанавливает 10 единиц корпуса за деньги и со штрафом к mood экипажа.
- После её личного квеста и флага mechanic_aboard корабль получает ремонт на месте, слабее эффекты поломок и уникальные модули.
- Скидки на модули и цена улучшений зависят от mechanic.attitude через условие requires и поле discount.

Начальница. Её отдельный «романтичный полёт» это особый заказ с флагом boss_aboard, который открывается при высоком boss.attitude и добавляет её в crew_aboard на один рейс.

Визуал космоса: 3D-модели, вид сверху. Полёт и бой показываются камерой сверху на низкополигональных 3D-моделях, а вся логика остаётся на плоскости. Так не нужны спрайты под разные ракурсы, поворот корабля и свет работают сами. В данные кораблей, врагов и NPC добавляются поля модели, логика их не читает:

```
{
  "id": "pirate_light",
  "model": "res://art/models/ships/pirate_light.glb",
  "model_scale": 1.0,
  "model_yaw_offset": 0,
  "engine_fx": ["engine_glow_small"]
}
```

- Камера смотрит строго вниз или под большим углом. В полёте она следует за кораблём игрока, в бою фиксирована над полем, сетка рисуется плоскостью.
- Позиции, дальность и углы обзора считаются по 2D-координатам (x и z). Корабли движутся по плоскости y = 0, модель только вращается вокруг вертикальной оси.
- Правила для моделей: формат glTF (.glb), один масштаб (1 единица равна одной клетке сетки), одинаковое направление «вперёд», мало материалов и несколько тысяч треугольников на корабль.
- Веб-билд: насколько я знаю, веб-экспорт Godot 4 работает в режиме совместимости, поэтому держи простые материалы и мало источников света, и рано проверь веб-сборку с тремя-пятью кораблями.
- Хаб, корабль изнутри и героини остаются в 2D (фон-картинка и слои спрайта). 3D только в сценах system_flight и combat.

- Пока моделей нет, DataLoader подставляет простую заглушку (конус или куб), так что логику можно тестировать до появления арта.

Бой

Бой пошаговый, в духе Rogue Trader: сетка, позиция и направление корабля, очки действий. Сначала делай минимум: один тип врага и три действия.

```
{
  "id": "pirate_light",
  "name": "Пиратский катер",
  "hull": 30,
  "shield": 10,
  "speed": 3,
  "weapons": [{ "id": "cannon_s", "dmg": [4, 7], "range": 4, "arc": "front"
}],
  "ai": "aggressive",
  "tags": ["pirate"],
  "can_negotiate": true,
  "loot": { "money": [100, 300], "fuel": [0, 2] }
}
```

```
{
  "id": "evasive_maneuver",
  "owner": "pilot",
  "name": "Манёвр уклонения",
  "cost_ap": 2,
  "cooldown": 3,
  "effect": { "self_evasion": 0.4, "duration": 2 },
  "mood_scale": { "stat": "pilot.mood", "min": 0.75, "max": 1.25 }
}
```

Героиня	Способности
Пилот	evasive_maneuver (уклонение), emergency_burn (рывок ценой топлива)
Безопасница	focus_fire (усиленный залп), repel_boarding (отражение абордажа)
Медик	field_repair_crew (лечение команды), calm_aura (поднимает mood)
Механик	hull_patch (ремонт корпуса), доступна только при флаге mechanic_aboard

Ход боя (стартовые значения): 6 очков действий на ход. Игрок тратит их на манёвр, оружие, щиты и способности. Оружие бьёт по сектору обзора, поэтому важно направление. Затем ход врагов по ai.

- Сила способности умножается на mood_scale: при хорошем настроении героини эффект выше, при плохом ниже.
- Альтернативы бою: побег (топливо и шанс пилота), переговоры (если can_negotiate, зависят от репутации и тегов), взятка (деньги).
- Итоги боя применяются тем же EffectApplier: лут, повреждения, репутация, симпатии.

Встреча в системе: от полёта к бою

1. В system_flight патрули, торговцы и пираты летают по своим маршрутам в реальном времени, как и корабль игрока.
2. Когда чужой корабль входит в радиус встречи, полёт ставится на паузу и открывается диалоговое окно с вариантами события.
3. Только вариант с start_combat загружает боевое поле. Остальные варианты (откупиться, договориться, уйти) заканчиваются без боя, и полёт продолжается.
4. После боя корпус, топливо и лут возвращаются в GameState, а побеждённый корабль убирается из системы или улетает.

```
{
  "id": "npc_patrol_03",
  "faction": "state",
  "behavior": "patrol",
  "route": ["alpha_1", "gate_alpha_beta"],
  "speed": 2,
  "encounter": { "radius": 120, "event": "patrol_check" }
}
```

Движение и дальность в бою

Боевое поле это сетка. Стартовые позиции берутся из взаимного расположения кораблей в момент встречи.

- Очки движения и очки действий считаются отдельно. Стартовое значение: move_points = engine.thrust − floor(масса груза ÷ 4), минимум 1. Двигатель это модуль со слотом engine и полем thrust (например, engine_mk1 с thrust 4). Зона движения и инерция описаны в подразделе «Движение в бою».
- Оружие описывается типом, дальностью и точностью. Точность линейно падает от close до far, за пределами range выстрел невозможен.

```
{
  "id": "cannon_s",
  "type": "kinetic",
  "dmg": [4, 7],
  "range": 4,
  "ap_cost": 2,
  "arc": "front",
  "accuracy": { "close": 0.9, "far": 0.5 }
}
```

Тип	Урон	Особенности
kinetic	по корпусу 100%, по щитам 50%	средняя дальность
laser	по щитам 100%, по корпусу 75%	большая дальность, точность падает слабо
missile	высокий урон	ограниченный боезапас, дальность большая, точность низкая вблизи цели

Способность пилота emergency_burn добавляет очки движения на ход, а тяжёлый груз их забирает, поэтому состав груза влияет на бой.

Движение в бою: конус и инерция

Корабль не летает в любую клетку вокруг себя. Он имеет курс (8 направлений), не может идти назад и выбирает клетку внутри конуса, который расширяется вперёд. Навык манёвра расширяет конус. Ход можно потратить несколькими перемещениями, и каждое сужает конус в сторону, противоположную уже сделанному сдвигу: так передаётся инерция.

Для клетки считаются две величины относительно курса в начале хода: f это сколько клеток вперёд, l это сколько вправо (влево отрицательное). Для Godot (x вправо, z вниз) при курсе на север h = (0, −1), правая сторона равна (−h.z, h.x).

```
k      = min(1.0, 0.5 + 0.12 × maneuver_level)     наклон конуса
W(f)   = 0.5 + k × f                               полуширина конуса на f
клеток вперёд
c      = max(0.1, 0.6 − 0.1 × maneuver_level)      коэффициент инерции
L      = смещение вправо от начала хода (влево отрицательное)
left_w  = max(0, W(f) − c × max(L, 0))
right_w = max(0, W(f) − c × max(−L, 0))

клетка доступна, если:
  f ⩾ 1
  −left_w ⩽ l ⩽ right_w
  ceil(расстояние до клетки) ⩽ оставшиеся очки движения
  клетка свободна и путь до неё не проходит через препятствия
```

- Курс во время хода не меняется. После каждого перемещения конус пересчитывается от новой позиции, f и l считаются от неё, а L от позиции начала хода.
- В конце хода курс поворачивается в сторону суммарного смещения не больше чем на 1 + floor(maneuver_level ÷ 2) шагов по 45°. Если корабль не двигался, курс не меняется.
- Игрок видит подсветку доступных клеток и может отменить последнее перемещение, пока не нажал «завершить ход».
- Враги используют ту же функцию, чтобы выбирать клетки.

```
"movement": {
  "maneuver_level": 0,
  "cone_base": 0.5,
  "cone_slope_base": 0.5,
  "cone_slope_per_level": 0.12,
  "cone_slope_max": 1.0,
  "inertia_base": 0.6,
  "inertia_per_level": 0.1,
  "inertia_min": 0.1,
  "turn_steps_base": 1
}
```

Пример для проверки (8 очков движения, курс вверх): при maneuver_level 0 доступны 38 клеток, при уровне 3 доступны 50. Если корабль ушёл на 3 вперёд и 2 вправо (потратил 4 очка), то из оставшихся 4 очков движения доступны 8 клеток, а без инерции было бы 12: клетки (0,6), (1,4), (1,5), (1,6) относительно начала хода становятся недоступны. Все числа стартовые, баланс подбирается тестами.

Порядок вайбкода

Каждый этап заканчивается запускаемой версией и коммитом в Git. Следующий этап начинается только после проверки предыдущего.

Этап	Результат	Как проверить
1. Скелет	GameState, DataLoader, Clock, пустые сцены	игра запускается, JSON читается без ошибок
2. Диалоги	dialogue-сцена, ConditionEvaluator, EffectApplier, тестовая героиня и 3 сцены-заглушки	шкалы меняются после выбора, условия закрывают и открывают сцену
3. Хаб	локации, расписание, смена слотов суток	героини переходят по расписанию, сцены зависят от места и времени
4. Заказы	ПК, 6 шаблонов заказов, репутация	заказ берётся, награда и провал меняют деньги и репутацию
5. Карта	граф систем, неизвестные маршруты, топливо	диапазон расхода виден, настоящий расход открывается после прохода

Этап	Результат	Как проверить
6. Полёт	system_flight в реальном времени, NPC-корабли, диалог при встрече	подлёт к маркеру запускает событие с вариантами
7. Бой	один враг, движение по сетке, три действия, способности пилота	бой заканчивается лутом или провалом без тупика
8. Стелс, магазин, готовка	stealth_minigame, items.json, shops.json, recipes.json, кухня и магазин в хабе	зона стелса растёт с успехами, блюдо готовится из купленных ингредиентов и подаётся героине
9. Контент	сюжет и сцены героинь по готовому сценарию, секретность, арт слоями	вертикальный срез: один рейс от заказа до вечерней сцены

Стартовый промпт для нейронки (подставь нужный этап):

```
Ты помогаешь делать игру на Godot 4: только GDScript, без C#. Визуальная новелла плюс космические полёты. Игра должна экспортироваться в Web (режим совместимости).
Правила проекта:
1. Весь контент (сцены, героини, заказы, маршруты, враги, предметы, рецепты) хранится в JSON в res://data. Тексты игры в коде не пишутся.
2. Все условия проверяет ConditionEvaluator, все последствия применяет EffectApplier. Новые типы добавляются только туда.
3. Всё состояние живёт в GameState (autoload), сохранение это его сериализация.
4. Делай одну задачу за раз, после неё игра должна запускаться. Не переписывай файлы, не относящиеся к задаче.
5. Ссылки между файлами только по id, DataLoader при старте проверяет, что все id существуют.
6. Пиши типизированный GDScript (var x: int), маленькие скрипты, один скрипт на одну ответственность.
7. Хаб, корабль изнутри и героини это 2D. Космос (system_flight, combat) это 3D-модели с камерой сверху, а логика считает позиции на плоскости x и z. Если модели нет, подставляй конус.
8. Не используй потоки и внешние процессы, чтобы веб-экспорт работал.
Задача: реализуй этап N из схемы данных (ниже вставлена схема) и покажи, как проверить результат.
```

Совет: перед каждым этапом вставляй в нейронку актуальную схему этого документа и только нужные ей JSON-файлы. Так она не изобретает свои поля.